Die Palettenreihe im Lager

Selection Sort — das Kleinste auswählen und nach vorn holen

AUSGANGSSITUATION



Ahmed macht sein Praktikum im Getränkelager. Acht Paletten stehen in einer Reihe, jede trägt eine **Tournummer**. Für die Auslieferung müssen sie aufsteigend geordnet stehen. Mit dem Hubwagen kann Ahmed immer nur zwei Paletten die Plätze tauschen lassen – jeder Tausch dauert **zwei Minuten**. Die Lagerleiterin: „Schauen kostet nichts, fahren kostet Zeit. Fahr so wenig wie möglich." Mit dem Nachbarn-Verfahren aus der letzten Stunde müsste Ahmed diese Reihe 15-mal tauschen.

Wie oft muss Ahmed höchstens tauschen, wenn er geschickt vorgeht – und braucht er dafür auch weniger Vergleiche als beim Bubble Sort?

Was tippst du? Kreuze an, bevor du sortierst:

- ☐ weniger Tauschvorgänge und weniger Vergleiche
- ☐ weniger Tauschvorgänge, gleich viele Vergleiche
- ☐ gleich viele Tauschvorgänge

Die Palettenreihe vor dem Sortieren – Tournummern (gesetzte Annahme)

Platz	0	1	2	3	4	5	6	7
Tournummer	41	17	58	23	66	12	35	29

1 Aufgabe 1 — Aufwärmen

Nenne aus dem Gedächtnis die Anzahl der Vergleiche, die Bubble Sort aus der letzten Stunde bei acht Elementen macht.

SO GEHT'S

So sortiert Ahmed mit wenig Fahrten: Selection Sort

1. Der **sortierte Teil** links ist am Anfang leer, der unsortierte Teil ist die ganze Reihe.
2. Suche im unsortierten Teil das **kleinste** Element – durch Anschauen, von links nach rechts.
3. Tausche es mit dem ersten Element des unsortierten Teils. Steht es schon dort, ist kein Tausch nötig.
4. Der sortierte Teil ist um eins gewachsen. Wiederhole ab Schritt 2, bis nur noch ein Element übrig ist.

BEISPIEL

Beispiel mit vier Zahlen

7 3 9 1: Durchgang 1: kleinstes ist 1 (Platz 3), Tausch mit Platz 0 → **1** 3 9 7 (3 Vergleiche, 1 Tausch). Durchgang 2: kleinstes von 3 9 7 ist 3 – steht schon vorn, kein Tausch (2 Vergleiche). Durchgang 3: kleinstes von 9 7 ist 7, Tausch → **1 3 7** 9. Fertig: 6 Vergleiche, 2 Tauschvorgänge.

2 Aufgabe 2

Vervollständige die Tabelle: Trage je Durchgang die kleinste Nummer des unsortierten Teils, die Reihe danach und ein, ob getauscht wurde. Der erste Durchgang ist eingetragen.

Durchgang	kleinste	0	1	2	3	4	5	6	7	Tausch?
Start	–	41	17	58	23	66	12	35	29	–
1	12	12	17	58	23	66	41	35	29	ja
2										
3										
4										
5										
6										
7										

Tauschvorgänge insgesamt: _____ Vergleiche insgesamt (7 + 6 + … + 1): _____

3 Aufgabe 3

Erkläre, warum Selection Sort in jedem Durchgang höchstens einmal tauscht – obwohl es im Durchgang viele Vergleiche gibt.

Satzanfänge: „Beim Vergleichen merkt sich Ahmed nur …“ · „Erst wenn …, dann …“

MERKE

Zwei Arten von Arbeit

Ein Sortierverfahren verursacht **Vergleiche** (schauen) und **Tauschvorgänge** (bewegen). Bubble Sort und Selection Sort vergleichen gleich oft: $n \cdot (n - 1) / 2$. Beim Tauschen unterscheiden sie sich: Selection Sort tauscht höchstens $n - 1$ mal, Bubble Sort bei jedem Vergleich, bei dem der linke Wert größer ist – im schlimmsten Fall ebenfalls $n \cdot (n - 1) / 2$ mal.

4 Aufgabe 4

Bestimme für Ahmeds Palettenreihe die Vergleiche, die Tauschvorgänge und die Fahrzeit (zwei Minuten je Tausch) – für Bubble Sort aus der letzten Stunde und für Selection Sort aus Aufgabe 2.

Bubble Sort: Vergleiche _____ Tauschvorgänge _____ Fahrzeit _____ min

Selection Sort: Vergleiche _____ Tauschvorgänge _____ Fahrzeit _____ min

5  Aufgabe 5 — an den Rechner

Algorithmen-Werkstatt → Selection Sort → **Selbst machen**. Du klickst in jedem Durchgang das kleinste Element des unsortierten Teils an – die Website tauscht und zählt mit.

`https://philipp-byte.github.io/algorithmen-werkstatt/selection-sort.html`

Dokumentiere einen Sortierlauf mit einer Zufallsliste: deine Startreihe, die Tauschvorgänge und die Vergleiche.

Startreihe	Tauschvorgänge	Vergleiche

INFO

Das Kleinste nur merken

Der Code bewegt beim Suchen nichts. Er merkt sich nur den **Index** des bisher kleinsten Elements in der Variablen `kleinster`. Die innere Schleife beginnt hinter dem Anfang des unsortierten Teils: `range(start + 1, n)` liefert die Indizes von start + 1 bis $n - 1$.

Erst nach der inneren Schleife wird einmal getauscht – mit `a, b = b, a` wie in der letzten Stunde.

6 Aufgabe 6

Implementiere die fehlenden Teile der Funktion. Sie soll die Liste aufsteigend sortieren und zurückgeben.

```
1  def selection_sort(liste):
2      n = len(liste)
3      for start in range(n - 1):
4          kleinster = ________
5          for i in range( _________ , n):
6              if liste[i] _____ liste[kleinster]:
7                  kleinster = ______
8          liste[start], liste[kleinster] = liste[kleinster], liste[start]
9      return liste
10
11 paletten = [41, 17, 58, 23, 66, 12, 35, 29]
12 print(selection_sort(paletten))
```

Wenn du nicht weiterkommst: Welches Element ist zu Beginn eines Durchgangs das „bisher kleinste“ – bevor irgendetwas verglichen wurde?

7 Aufgabe 7 — am Rechner

Teste deine Funktion in der Werkstatt unter **Code schreiben** (oder in Thonny) mit der Palettenreihe. Trage die ersten drei und die letzte Nummer der Ausgabe ein und kreuze an, wenn alle Tests bestanden sind.

Ausgabe beginnt mit: _____ _____ _____ … und endet mit: _____

☐ alle Tests bestanden



Diese Aufgabe hat mehr als eine richtige Antwort – es kommt darauf an, **was im Lager teuer ist**. Begründe deine Wahl mit den Zahlen aus Aufgabe 4.

8 Aufgabe 8 — Ahmeds Entscheidung

Beurteile mit deinen Zahlen aus Aufgabe 2 und 4, welches Verfahren Ahmed nehmen soll – und ob er damit auch beim Schauen Zeit spart. Vergleiche am Ende mit deinem Tipp vom Anfang.

Satzanfänge: „Ahmed sollte …, weil er dann nur … statt … tauscht." · „Beim Vergleichen spart er …, denn …"

9 Aufgabe 9 — Vertiefung

Übertrage das Verfahren auf die Bestenliste des Schulturniers: Die höchste Punktzahl soll auf Platz 0 stehen. Schreibe die Zeilen der Funktion auf, die sich ändern, und gib der Variablen einen passenden neuen Namen.

10 Aufgabe 10 — Zusatz

Entwickle einen Plan für das Lagerprogramm: Es verwaltet 3000 Artikelnummern, und jeden Tag wird etwa 500-mal geprüft, ob eine Nummer existiert. Halte fest, welche Verfahren aus den letzten vier Stunden das Programm in welcher Reihenfolge nutzen soll, und schätze die Vergleiche pro Tag mit und ohne deinen Plan.

PRÜFE DICH SELBST

- [ ] Wächst in deiner Tabelle der sortierte Teil links in jeder Zeile um genau eine Nummer?
- [ ] Ist deine Zahl der Vergleiche für Selection Sort dieselbe wie für Bubble Sort?
- [ ] Gibt deine Funktion 12 als erste und 66 als letzte Nummer aus?